

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

OpportunIQ is an AI-driven decision intelligence platform designed to help university students systematically discover, evaluate, and prioritise economic opportunities such as scholarships, internships, competitions, grants, and certifications.

In the current ecosystem, opportunity-related information is highly fragmented across informal channels (e.g., messaging groups, social media) and formal platforms (e.g., university portals, career websites). While access to opportunities is not inherently scarce, students face a critical bottleneck in information processing and decision-making.

OpportunIQ addresses this gap by transforming raw opportunity data into context-aware, prioritised, and explainable recommendations. Instead of acting as a passive listing system, the platform actively guides users toward high-impact decisions by identifying:

- *The most relevant opportunities aligned with their profile*
- *The urgency and potential value of each opportunity*
- *The required actions to maximise success probability*

This aligns strongly with the hackathon theme of AI-powered decision intelligence, where systems must go beyond automation and provide meaningful reasoning and actionable insights.

1.1. Purpose:

This System Analysis Documentation (SAD) outlines the technical architecture, system design, and decision intelligence mechanisms behind OpportunIQ.

So, the key element of system that has been covered are as followed:

1. Architecture:

- *A modern web-based architecture utilizing a thin frontend (Next.js) communicating with a robust Python backend (FastAPI) that orchestrates reasoning tasks via a dedicated LLM service layer.*

2. Data Flows:

- *How student profile context (CGPA, goals, readiness) merges with structured opportunity data, moves through the AI reasoning engine, and returns to the user as actionable, prioritized JSON objects.*

3. Model Process:

- *The end-to-end workflow of the core features: onboarding via the Smart Student Profile, processing through the Opportunity Intelligence Engine, and outputting actionable insights onto the Priority Dashboard.*

4. Role of Reference:

© UMHackathon 2026

Email: umhackathon@um.edu.my

- *This document serves as the central reference for the hackathon development team, UI/UX designers, and evaluating judges to understand the high-level architecture and logic flow of OpportunIQ.*

1.2. **Background:**

Malaysian university students, specifically at the University of Malaya (UM), lose real economic value every semester. Life-changing scholarships, internships, and grants are missed because the information is fragmented across WhatsApp groups, student portals, and career sites. The core problem is not a lack of opportunities, but decision overload and the failure to discover and prioritize them in time.

This results in lost economic value per semester, making the problem both practical and measurable.

1. **Previous Version:**

Currently, students rely on fragmented notice boards, university emails, and manual keyword filtering. These existing solutions can only retrieve listings based on simple fields (e.g., displaying all scholarships with an upcoming deadline) but cannot reason about which ones are actually worth the student's time based on their unique context.

2. **Changes in Major Architectural Components**

OpportunIQ introduces a generative AI reasoning engine (GLM) at the core of the system. A filter retrieves. OpportunIQ reasons. Without the GLM layer, the system collapses into a static list; with it, the system interprets messy text, compares trade-offs, calculates effort vs. economic value, and recommends immediate action.

1.3. **Target Stakeholder**

Stakeholders	Roles	Expectations
University Malaya Students	Student) Inputs profile data, browses recommended opportunities, and interacts with the AI advisor.	A personalized, actionable dashboard ("tell me which 3 are worth my time this week") instead of a massive list. Visibility into urgency and clear next steps.

<i>Universities</i>	<i>Provide opportunity data</i>	<i>Reliable integration and visibility</i>
<i>Sponsors / Organisations</i>	<i>Publish opportunities</i>	<i>Better reach to qualified students</i>
<i>Development Team</i>	<i>Build, maintain, and integrate the frontend, backend, and AI APIs.</i>	<i>Clear API contracts, modular codebase, low-latency prompt execution, and strict JSON output handling.</i>
<i>QA Team</i>	<i>Validating the system behavior for all the user roles</i>	<i>Defined scope, the request and response formats, and Defined input/output formats</i>

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (Browser-based)
Architecture	Client-Server with AI Service Layer

Architecture Description

OpportunIQ is structured as a modern web application separated into distinct layers to ensure speed and maintainability during the hackathon. The Next.js frontend handles the user interface and state. The FastAPI backend serves as the orchestration layer, receiving user data, retrieving curated opportunities from the database, and sending highly structured context to the Z.AI GLM API. The AI engine then processes this data and returns a scored, ranked, and reasoned JSON response back to the frontend.

2.1.2 LLM as Service Layer

The architecture explicitly isolates the Z.AI GLM as a central reasoning service, not just a generic chatbot add-on.

2.1.3 Dependency Diagram

Prompt Construction:

The FastAPI backend constructs a prompt by combining the user's Smart Student Profile (CGPA, goals, available assets like a CV) with a chunk of Curated Opportunity Records (eligibility, deadlines, value).

Context Window Management:

Data is formatted into strict dictionaries before injection to maximize token efficiency and prevent context overflow.

Parsing & Routing:

The GLM API is instructed to return a structured JSON response (via Pydantic models). The backend validates this JSON (Fit Score, Urgency Score, Estimated Value, Recommendation, Reasoning) before passing it to the frontend via REST API.

Token Limitation:

Input prompts are strictly limited to relevant opportunities only (pre-filtered by basic eligibility like course/year before hitting the LLM) to enforce token limits and maintain low latency.

2.2.2. Sequence Diagram

1. User submits Smart Profile (Frontend).
2. Frontend sends POST /api/analyze to FastAPI Backend.
3. Backend fetches raw curated opportunities matching baseline criteria.
4. Backend sends contextual prompt to Z.AI GLM API.
5. Z.AI GLM processes logic: $\text{Priority} = \text{Fit} + \text{Value} + \text{Urgency} + \text{Strategic Value} - \text{Effort} - \text{Risk}$.
6. Z.AI GLM returns structured JSON with categories: Apply Now, Prepare Soon, Track Later, Skip for Now.
7. Backend parses and serves formatted data to Frontend.
8. Frontend renders the Priority Dashboard.

2.2. Technological Stack

Layer	Technology	Justification
Frontend	Next.js (App Router), TypeScript, Tailwind CSS, shadcn/ui	Chosen for speed, clean production-style components, and solid routing
Backend	FastAPI (Python 3.10+), Pydantic models, Uvicorn	Chosen for typed endpoints, fast execution, and seamless

		AI/JSON integration
AI Engine	Z.AI GLM	Core reasoning capability

2.3. Key Data Flows

2.3.1.1. Data Flow Diagram (DFD)

Data originates from the user (Profile Setup) and flows into the FastAPI backend. Here, the system queries the internal curated Opportunity Dataset. Both sets of data are serialized and sent securely to the Z.AI GLM API. The AI processes the trade-offs and flows the data back to the backend as structured decision metrics (Scores, Economic Value, Action logic). Finally, the backend distributes this data to the Next.js frontend to populate the Opportunity Cards and Missed Value Tracker.

2.3.2. Normalized Database Schema

To ensure data integrity, the logical data model uses structured relationships:
 Students (ID, Course, Year, CGPA_Range, Goals)
 Opportunities (ID, Category, Estimated_RM_Value, Deadline, Required_Assets)
 Student_Readiness (Junction table linking students to assets like CVs, transcripts)

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. [e.g. in Table]

#	Features	Description
1	Smart Student Profile	2-minute onboarding capturing course, year, goals, time availability, and current asset readiness (CV, portfolio).
2	Explainable Opportunity Cards	Detailed cards showing Fit Score, Estimated RM Value, Urgency, Effort Level, and plain-language reasoning for the recommendation

3	Decision Scoring & Priority Dashboard	GLM analyzes opportunities against the profile and categorizes them into actionable buckets: Apply Now, Prepare Soon, Track Later, Skip for Now.
4	Missed Value Tracker	A dashboard widget quantifying the user's pipeline (e.g., "You have RM12,400 of relevant opportunities. RM3,500 may expire in 5 days.").
5	AI Advisor Chat	Context-aware chat allowing users to ask trade-off questions (e.g., "Which of these should I prioritize for immediate ROI?").

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g.)
Scalability	Must handle concurrent requests from judges and test users without bottlenecking the AI engine.	Fast concurrent request handling using FastAPI's async endpoints and Uvicorn workers.
Reliability	The app must not break if the GLM API fails or returns malformed text.	Pydantic validators strictly enforce JSON schema. Fallback logic defaults to a standard chronological list if AI scoring fails.
Maintainability	Keep frontend logic thin; all complex reasoning and orchestration must live in the backend.	Clear folder separation (backend/ for Python, react-router-app/ for UI).
Token Latency	GLM reasoning should not leave the user staring at a blank screen.	Async API calls with an enforced timeout. Frontend implements skeleton loaders and streaming UI states while awaiting the AI payload.
Cost Efficiency	Prevent token bloat during the hackathon validation phase.	Implement a pre-filtering logic layer in Python to drop highly irrelevant opportunities before passing the list to the GLM for deep reasoning.

3.3. Out of Scope / Future Enhancements

These features are part of the grand vision for a national multi-university platform but are excluded from the current hackathon sprint:

- 1. Live web scraping of unverified internet sources (Relying strictly on a curated, verified dataset for the pilot).*
- 2. Direct application submission integrations with third-party scholarship portals.*
- 3. Full institutional backend portals for university admins to upload data.*
- 4. Persistent user accounts using full Auth (e.g., Supabase/Clerk) to save time during the hackathon demo flow.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

Grayscale Rollout

- Initial deployment to a small user group to monitor performance before scaling.

A/B Testing

Used to evaluate:

- Recommendation formats
- Prioritisation strategies

4.1.2. Emergency Rollback (ER) & Golden Release:

Emergency Rollback

- Fallback to stable version if:
 - AI fails
 - API latency spikes

4.1.3. Priority Matrix:

Priority	Condition	Action
----------	-----------	--------

P1	Z.AI GLM API rate limit reached or timeout.	Graceful degradation. The UI notifies the user and reverts to a basic filter system (displaying opportunities by deadline) rather than crashing.
P2	Frontend fails to parse JSON due to hallucinated keys	Pydantic models in FastAPI strip out unknown keys and enforce defaults before it hits the frontend

4.2. **Monitoring:**

Internal event logging within FastAPI will monitor the time taken for Z.AI GLM to return outputs. Errors will be printed to the backend terminal for live debugging during the hackathon presentation.

4.3. **Assumptions:**

Target users (UM Students) know their basic academic metrics (CGPA, credit hours) and can accurately input their readiness level.

The curated dataset of opportunities (scholarships, grants) pre-loaded into the system is accurate and verifiable.

4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	Core reasoning engine for ranking, scoring, plain-language explanations, and chat.	High – If the API goes down or hits rate limits, decision intelligence is lost. Mitigation: Implement strict error handling and basic fallback sorting.
shadcn/ui & Tailwind	Rapid frontend styling and component generation.	Low – Bundled securely in the Next.js app.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Phase	Duration
Day 1-2 (Planning & Setup)	Finalize dataset, wireframe UI, initialize Next.js and FastAPI repos.
Day 3-5 (Backend & AI Logic)	Build FastAPI endpoints, write Pydantic schemas, perfect prompt engineering with Z.AI GLM API to ensure stable JSON output.
Day 6-8 (Frontend Integration)	Build out the Priority Dashboard, hook up UI components to the API, implement loading states and AI Chat.
Day 9-10 (Polish & Pitch)	Finalize the "Missed Value Tracker" economic framing, stress-test the user journey, and prepare the demo narrative for judges.

5.2. Team Members Role:

Member	Role
Backend & AI Lead	Responsible for FastAPI architecture, prompt engineering, GLM API integration, and mathematical scoring logic.
Frontend Lead	Responsible for Next.js, shadcn components, state management, and the user onboarding flow.
Product/Data Lead	Curates the verified UM dataset, defines the economic empowerment narrative, and prepares the hackathon pitch/slides

5.3.

5.4. **Recommendations:**

To ensure a flawless demo, strictly separate the frontend from the backend. Keep the Next.js app entirely focused on UI state and let FastAPI handle all heavy lifting, prompt orchestration, and JSON cleaning. This ensures the demo is snappy, credible, and perfectly highlights the GLM's role as a true decision intelligence system.